JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 : 202118045 **이름:** 전민상 **제출일:** 2026-10-28

서비스 이름: 강의 수강 후기 게시판

기술 조합과 버전: Vue 3.5.43 (Vite 8.3.4) / Spring Boot 3.5.16 (JJWT 0.13.0, SQLite JDBC 3.51.3.0) / Windows 10, Node.js v24.22.1, JDK Temurin 21.0.12.1

GitHub 저장소 URL https://github.com/wjsalstkd08/202118045_course_review

소스 코드 기준 main / e4c57e0

수강신청을 앞둔 학생이 강의의 수업 방식과 과제·시험 부담을 미리 확인할 수 있도록 수강 후기를 공유하는 게시판이다. 누구나 로그인 없이 후기를 읽을 수 있고, 로그인한 사용자만 후기를 작성할 수 있으며, 자신이 쓴 후기만 수정·삭제할 수 있다. 과목명은 제목에 [과목명] 형식으로 적는다.

기술 조합은 Vue 와 Spring Boot 를 선택했다. 이전 실습에서 IntelliJ 로 Spring Boot 프로젝트를 실행해 본 경험이 있어 백엔드 실행 환경이 익숙했고, 7 주차 강의자료가 Spring Boot 와 JJWT 로 JWT 를 발급·검증하는 방법을 다루고 있어 참고하기 좋았기 때문이다. 강의자료는 Spring Security 의 JwtFilter 를 사용하지만, 제공 템플릿은 Security 없이 Controller 가 AuthService 를 직접 호출하는 구조여서 템플릿 구조를 유지했다. 그래야 명세가 요구하는 JSON 오류 형식(TOKEN_REQUIRED·INVALID_TOKEN·TOKEN_EXPIRED 구분)을 별도 설정 없이 맞출 수 있다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.list / get (제공 코드)	토큰 없이 조회 가능
로그인과 JWT 발급	완료	AuthService.issueToken, 생성자 / api.js login	HS256, sub·iss·aud·iat·exp, TTL 90 초
인증과 내 정보 조회	완료	AuthService.currentUserId / BoardController.me	401 code 3 종 구분
본인 글 작성·수정·삭제	완료	AuthService.assertOwner / api.js canEdit	작성자는 서버가 토큰으로 결정, 타인 글 403
만료·로그아웃·오류 표시	완료	api.js request / App.vue run·logout	401 시 세션 해제, 404 시 목록 갱신
DB 데이터 유지	완료	PostStore (제공 코드), backend-spring/data/board.sqlite	서버 재시작 후 유지

구조와 변경 내역

[브라우저 (Vue, 127.0.0.1:4173)] → /api 요청 → [Vite preview 프록시] → [Spring Boot API (127.0.0.1:8000)] → [SQLite 파일 data/board.sqlite]

로그인 요청: 로그인 폼 → api.js login() → POST /api/auth/login → BoardController.login → Users.verify(비밀번호 확인) → AuthService.issueToken → 응답 {accessToken, expiresIn, user} → api.js 가 토큰과 user 를 모듈 변수 session 에 저장(메모리). 이 요청에는 Authorization 헤더를 보내지 않는다.

글 수정 요청: 수정 저장 버튼 → api.update() → request()가 authorizationHeaders()로 'Authorization: Bearer <JWT>' 첨부 → PUT /api/posts/{id} → BoardController.update → AuthService.currentUserId(토큰 검증) → PostStore.get(글 존재) → AuthService.assertOwner(작성자 비교) → 입력 검증 → PostStore.update → 200 응답.

토큰 저장 위치는 브라우저 메모리(api.js 의 session 변수)뿐이며 localStorage·sessionStorage 를 쓰지 않는다. 토큰은 /me, 글 등록·수정·삭제 요청에 첨부되고, 토큰이 없으면 헤더 자체를 만들지 않는다. 새로고침하면 session 이 초기화되어 로그인 상태가 해제된다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-spring/.../AuthService.java / 생성자(추가)	없음	app.jwt.* 설정 주입, JWT_SECRET 이 비었거나 예시 값이거나 64 자리 16 진수가 아니면 서버 시작 거부, 32 바이트 키로 변환	BE-01 2 번: 키 검증과 32 바이트 변환. 시작 시 거부하기 위해 생성자에 둠 / AI 초안 후 본인 수정
AuthService.java / issueToken	501 NOT_IMPLEMENTED 반환	HS256 으로 sub·iss·aud·iat·exp 를 담은 JWT 발급	BE-01 / AI 초안 후 본인 수정
AuthService.java / currentUserId	데모 ID 1L 반환	Bearer 추출 → JJWT 검증(서명·알고리즘·만료·iss·aud) → sub 형식·계정 존재 확인 → 사용자 ID 반환, 실패 시 401 code 3 종	BE-02 / AI 초안 후 본인 수정
AuthService.java / assertOwner	검사 없음(빈 메서드)	현재 사용자와 글 작성자가 다르면 403 FORBIDDEN	BE-03 / AI 초안 후 본인 수정
frontend-vue/src/api.js / authorizationHeaders, request, login, canEdit	헤더 없음, 로그인 미구현, 모든 글 수정 가능 표시	토큰이 있을 때만 Bearer 헤더, 401 시 clearSession, 로그인 응답 저장, 작성자 비교	FE-01~04 / AI 초안 후 본인 수정
frontend-vue/src/App.vue / run, 템플릿 문구	오류 메시지만 표시, 데모 배너	404 시 선택 해제·목록 갱신, 데모 배너와 제목을 서비스 안내로 교체	FE-03(404), FE-04(배너) / 404 처리는 AI 초안, 화면 문구는 본인 작성

로컬 실행 기록

운영체제 Windows 10 / Node.js v24.22.1 / JDK Temurin 21.0.12.1 / Chrome 156.

백엔드: backend-spring 폴더에서 PowerShell 로 $env:JWT_SECRET(직접 생성한 64 자리 16 진수), $env:TOKEN_TTL='90' 설정 후 .₩mvnw.cmd spring-boot:run.

프론트: frontend-vue 폴더에서 npm ci → npm run build → npm run preview. SQLite 파일: backend-spring/data/board.sqlite(자동 생성).

http://127.0.0.1:4173 빌드본에서 목록 조회·로그인·등록·삭제가 동작함을 확인했다(GET 200, 로그인 200, 등록 201, 삭제 204). 서버를 Ctrl+C 로 종료 후 재실행해도 글·작성자·작성 시각이 유지되었다.

clone 재현: GitHub 저장소를 〈바탕화면₩clone-test〉에 새로 clone 했다. backend-spring 에서 환경변수를 설정하고 .₩mvnw.cmd package(BUILD SUCCESS) → java -jar target₩jwt-board-1.0.0.jar 로 서버를 실행했다. frontend-vue 에서 npm ci → npm run build → npm run preview 를 실행한 뒤 4173 에 접속해 로그인, 등록, 삭제가 동작함을 확인했다. 새 clone 이라 DB 파일은 clone-test₩backend-spring₩data₩board.sqlite 에 새로 생성되었고 초기 목록은 0 개였다.

증거: check_api.png, check_api_expiry.png, build_log.png, preciew_4173.png, clone_test.png, clone_test_4173.png

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-spring/src/main/java/edu/classroom/board/AuthService.java / 생성자, issueToken (호출: BoardController.login)

코드 발췌

```
public AuthService(
        @Value("${app.jwt.secret}") String secret,
        @Value("${app.jwt.issuer}") String issuer,
        @Value("${app.jwt.audience}") String audience,
        @Value("${app.jwt.ttl}") long ttl,
        Users users) {
  if (secret == null || secret.isBlank() || secret.startsWith("REPLACE_WITH"))
    throw new IllegalStateException("JWT_SECRET 을 새 난수 키로 설정하세요.");
  if (!secret.matches("[0-9a-fA-F]{64}"))
    throw new IllegalStateException("JWT_SECRET 은 64 자리 16 진수여야 합니다.");
  this.key = Keys.hmacShaKeyFor(HexFormat.of().parseHex(secret)); // 64 자리 hex → 32 바이트 키
  this.issuer = issuer;
  this.audience = audience;
  this.ttl = ttl;
  this.users = users;
}
public String issueToken(Users.PublicUser user) {
  Instant now = Instant.now();
  return Jwts.builder()
          .subject(String.valueOf(user.id()))
          .issuer(issuer)
          .audience().add(audience).and()
          .issuedAt(Date.from(now))
          .expiration(Date.from(now.plusSeconds(ttl)))
          .signWith(key, Jwts.SIG.HS256)
          .compact();
}
```

동작과 응답 코드 설명

입력: 로그인 요청의 email·password. 형식 검사(문자열, 길이)는 제공 코드 BoardController.fields 가 하고, 틀리면 400 BAD_INPUT 이다. 계정 확인은 제공 함수 Users.verify 가 PBKDF2 해시로 비교하며, 틀린 비밀번호와 없는 이메일 모두 null 을 돌려줘 401 INVALID_CREDENTIALS 가 된다(두 경우 응답이 같아 계정 존재 여부가 드러나지 않음, T03).

처리: 계정 확인을 통과한 사용자에게만 issueToken 이 호출된다. sub 에는 사용자 ID 를 문자열로, iss·aud 에는 설정값(webservice-class, board-api)을, exp 에는 발급 시각 + TOKEN_TTL(기본 90 초)을 넣고 서버 키로 HS256 서명한다. 비밀번호와 해시는 토큰에 넣지 않는다.

키 처리: 서명 키는 생성자에서 한 번 만든다. Spring 이 서버 시작 시 AuthService 를 생성하므로, 키가 비었거나 예시 값이거나 64 자리 16 진수가 아니면 예외로 서버 시작 자체가 실패한다. 통과하면 16 진수 문자열을 32 바이트로 바꿔 HMAC 키를 만든다.

출력: 200 과 {accessToken, expiresIn: 90, user: {id, name, email}}. 실제 발급 토큰의 payload 는 {"sub":"1","iss":"webservice-class","aud":["board-api"],"iat":...,"exp":...}였고 exp − iat = 90 초였다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-vue/src/api.js / authorizationHeaders, request, login (화면: App.vue run·logout)

코드 발췌

```
function authorizationHeaders() {
    return session.accessToken
      ? { Authorization: `Bearer ${session.accessToken}` }
      : {};
}
```

```
if (!response.ok) {
  if (response.status === 401) clearSession();
  const error = new Error(
    data.message || data.code || `HTTP ${response.status}`,
  );
  error.status = response.status;
  error.code = data.code;
  throw error;
}
return data;
```

동작과 응답 코드 설명

로그인 응답 처리: login()은 먼저 이전 세션을 비워, 다른 사용자로 다시 로그인할 때 이전 토큰이 로그인 요청에 붙지 않게 한다. request()가 실패(401·400)하면 예외가 던져져 session 저장 줄에 도달하지 않으므로, 실패 시 로그인 화면으로 넘어가지 않는다. 성공하면 토큰과 user 를 모듈 변수 session 에만 저장한다.

전송: 모든 API 호출은 request()를 거치고, authorizationHeaders()가 토큰이 있을 때만 'Authorization: Bearer <JWT>'를 만든다. 토큰이 없으면 빈 객체를 돌려 'Bearer undefined' 같은 헤더가 생기지 않는다. 공개 목록·상세는 토큰 없이도 200 으로 조회된다(T01).

401 처리: 응답이 401 이면 clearSession()으로 토큰과 사용자를 지운다. App.vue 의 run()은 finally 에서 user 를 getSession().user 로 다시 읽기 때문에, 401 직후 화면이 '로그아웃 상태'로 바뀐다(T10). 403 은 세션을 지우지 않아 로그인이 유지된다. 404 는 App.vue run()에서 선택을 해제하고 목록을 다시 불러온다(T14).

로그아웃: App.vue logout()이 clearSession()을 호출해 이 탭의 토큰을 지운다. 이후 요청에는 Authorization 헤더가 없어 401 TOKEN_REQUIRED 가 된다(T11). 이는 서버에서 이미 발급된 토큰을 무효화하는 것이 아니며, 복사된 토큰은 exp 까지 유효하다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: AuthService.java / currentUserId (호출: BoardController.me·create·update·delete)

코드 발췌

```
public long currentUserId(HttpServletRequest request) {

  String header = request.getHeader("Authorization");
  if (header == null || !header.startsWith("Bearer ") || header.substring(7).isBlank())
    throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
  String token = header.substring(7).trim();

  Claims claims;
  try {
    Jws<Claims> jws =
          Jwts.parser()
                .verifyWith(key)
                .requireIssuer(issuer)
                .requireAudience(audience)
                .build()
                .parseSignedClaims(token); // 서명 검증 후 만료 검사
    claims = jws.getPayload();
    if (!"HS256".equals(jws.getHeader().getAlgorithm()) || claims.getExpiration() == null)
      throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  } catch (ExpiredJwtException e) {
    throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요.");
  } catch (JwtException | IllegalArgumentException e) {
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  }

  String sub = claims.getSubject();
  if (sub == null || !sub.matches("[1-9][0-9]{0,15}") || users.find(Long.parseLong(sub)) == null)
    throw new ApiError(401, "INVALID_TOKEN", "토큰의 사용자를 확인할 수 없습니다.");
  return Long.parseLong(sub);
}
```

동작과 응답 코드 설명

1 단계 헤더: Authorization 헤더가 없거나 'Bearer '로 시작하지 않거나 뒤가 비었으면 401 TOKEN_REQUIRED.

2 단계 검증: decode 로 내용을 읽기만 하지 않고 parseSignedClaims 로 검증한다. verifyWith(key)로 서버 키 서명을 확인하고, requireIssuer·requireAudience 로 iss·aud 를 비교하며, 서명 없는 토큰(alg=none)은 parseSignedClaims 가 거부한다. 추가로 헤더 알고리즘이 HS256 인지, exp 가 있는지 확인한다. JJWT 는 서명을 먼저 검증한 뒤 만료를 검사하므로, ExpiredJwtException 은 '서명은 정상이지만 만료된 토큰'일 때만 발생해 401 TOKEN_EXPIRED 로 구분된다. 서명 불일치·형식 오류·iss/aud 불일치 등 나머지 실패는 401 INVALID_TOKEN 이다.

3 단계 사용자: sub 가 양의 정수 문자열인지, 실제 계정(Users.find)이 있는지 확인한다. 통과한 ID 만 반환해 /me 와 등록·수정·삭제에 전달한다. 등록 시 작성자는 이 ID 로 서버가 정하므로 요청 본문의 값은 쓰이지 않는다.

응답: 실패는 ApiError 로 던져지고 제공 코드 Errors 가 {code, message} JSON 과 상태를 만들며, 401 에는 WWW-Authenticate: Bearer 를 붙인다. 테스트에서 서명 첫 글자만 바꾼 토큰은 /me·등록·수정·삭제 모두 401 INVALID_TOKEN(T09), TTL 3 초 토큰은 5 초 후 401 TOKEN_EXPIRED(T10)였다.

3 4 작성자 권한 검사

파일 / 함수: AuthService.java / assertOwner (호출: BoardController.update·delete)

코드 발췌

//BoardController.update(제공코드)

```
 public Object update(
    @PathVariable String id, @RequestBody Map<String, Object> body, HttpServletRequest req) {
  long user = auth.currentUserId(req), pid = postId(id);
  var p = existing(pid);
  auth.assertOwner(user, p.authorId());
  var input = fields(body, Map.of("title", 100, "content", 5000));
  return store.update(pid, input.get("title"), input.get("content"));
```

//AuthService.asertOwner

```
public void assertOwner(long userId, long authorId) {
    if (userId != authorId)
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.");
}
```

동작과 응답 코드 설명

비교 대상: 토큰 검증으로 얻은 현재 사용자 ID 와, DB 에 저장된 글의 authorId 를 비교한다. 요청 본문이나 화면에서 온 값은 비교에 쓰지 않는다.

판단 순서: Controller 는 인증(currentUserId) → 글 존재(existing) → 권한(assertOwner) → 입력 검증(fields) → DB 변경 순으로 호출한다. 그래서 토큰이 없으면 글이 없어도 401(T14), 인증된 Bob 이 Alice 글을 수정하면 403 FORBIDDEN 이고 DB 는 바뀌지 않는다(T07, T08). 같은 공백 제목이라도 Alice 본인은 400 BAD_INPUT, Bob 은 403 FORBIDDEN 으로 갈렸다(T13). 403 은 assertOwner 에서 예외로 끝나 store.update 까지 가지 않는다.

401 과 403 의 차이: 401 은 '누구인지 확인할 수 없음'(토큰 없음·변조·만료)이고, 403 은 '누구인지는 확인됐지만 이 글을 바꿀 권한이 없음'이다. 그래서 프론트는 401 에서만 로그아웃시키고 403 에서는 로그인을 유지한다.

작성자 위조 방지: 등록 시 authorId 는 currentUserId 결과로만 정해지고, 제공 코드 fields()가 title·content 외의 필드를 400 BAD_INPUT 으로 거부한다. Bob 이 authorId:1 또는 userId:1 을 넣어 보낸 요청은 400 이었고 글이 생성되지 않았다(T12). 화면의 버튼 숨김(api.js canEdit)은 편의 기능일 뿐이며, curl 로 직접 보낸 요청도 서버에서 막혔다.

테스트 결과

실행 환경: Windows 10 / Node.js v24.22.1 / JDK Temurin 21.0.12.1 / Chrome 156 / 백엔드 .\mvnw.cmd spring-boot:run, 프론트 npm run dev(5173) 및 npm run build → npm run preview(4173), API 직접 요청은 IntelliJ 내장 터미널 사용

로컬 배포 확인: npm run build 성공(dist/assets/index-C3wYzjPF.js) → npm run preview → 127.0.0.1:4173 에서 GET 200·로그인 200·등록 201·삭제 204 확인. (preview_4173.png)

테스트 일시 및 코드 버전: 2026-10-28 / e4c57e0

토큰 TTL: 일반 테스트 90 초 / 만료 테스트 3 초 (T10 및 check_api --expiry, TTL=3 으로 서버 재시작 후 수행)

05_테스트시나리오

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	GET/api/posts 로 목록 조회, 목록에서 글 클릭해 GET/api/posts/1 로 상세 조회 비로그인	200, 정상 공개 조회	목록 200 / 해당 없음, 상세 200 / 해당 없음. 요청에 Authorization 없음, 화면 "로그아웃 상태"에서 목록·상세 표시	PASS	T01_list.png T01_detail.png
T02	POST/api/auth/login Alice	200, JWT 와 공개 사용자 정보, 화면 로그인 표시	200 / 해당 없음. accessToken·expiresIn 90·user(id 1) 반환, 화면 “Alice 님 로그인 중”	PASS	T02_login_headers.png T02_login_response.png
T03	틀린 비밀번호, 이메일로 로그인 후 각각 POST/api/auth/login	401, INVALID_CREDENTIALS, 토큰 미발급	401 / INVALID_CREDENTIALS 401 / INVALID_CREDENTIALS. 응답에 토큰 없음, 화면 오류 메시지·”로그아웃 상태" 유지	PASS	T03_password_headers.png T03_password_response.png T03_email_headers.png T03_email_response.png
T04	“내 정보 조회”로 GET/api/me, 글 등록 후 POST/api/posts 비로그인	각각 401 TOKEN_REQUIRED	/me 401 / TOKEN_REQUIRED, 등록 401 / TOKEN_REQUIRED. 화면 "로그인이 필요합니다.", 목록 개수 변화 없음	PASS	T04_me_headers.png T04_me_response.png T04_write_headers.png T04_write_response.png
T05	POST /api/posts 로 글 등록 후 "내 정보 조회" GET /api/me Alice	201, authorId 1, /me 는 Alice	등록 201 / 해당 없음(Location: /api/posts/2, authorId 1), /me 200 / 해당 없음(id 1, Alice). 요청에 Bearer 토큰 첨부	PASS	T05_me_headers.png T05_me_response.png T05_write_headers.png T05_write_response.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T06	본인 글 PUT /api/posts/3 로 수정 후 DELETE /api/posts/3 Alice	200 및 204, 결과가 DB 에 반영	수정 200 / 해당 없음(content 변경, createdAt 유지), 삭제 204 / 해당 없음(본문 없음). 화면 목록 2 개→1 개	PASS	T06_update_headers.png T06_update_response.png T06_delete_headers.png T06_delete_response.png
T07	Alice 글 PUT /api/posts/1, 전후로 GET /api/posts/1 조회 Bob	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN. 재조회 200, 글 유지	PASS	T07_bob_update.png
T08	Alice 글 DELETE /api/posts/1 후 GET /api/posts/1 Bob	403 FORBIDDEN, 글 유지	403 / FORBIDDEN. 재조회 200, 글 유지	PASS	T08_bob_update.png T08_bob_delete.png
T09	Alice 토큰의 서명 첫 글자 변조(curl). GET /api/me, POST /api/posts, PUT·DELETE /api/posts/1, 전후 GET /api/posts	401 INVALID_TOKEN, 데이터 변경 없음	/me·등록·수정·삭제 모두 401 / INVALID_TOKEN (WWW-Authenticate: Bearer). 전후 목록 동일, 새 글 없음	PASS	T09_update1.png T09_update2.png
T10	TOKEN_TTL=3 재시작. Alice·Bob 토큰 발급 5 초 후 GET /api/me(Alice), PUT /api/posts/1(Bob). 브라우저에서 로그인하고 대기 후 내 정보 조회	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	/me 401 / TOKEN_EXPIRED, Bob 수정 401 / TOKEN_EXPIRED, 재조회 변경 없음. 화면 만료 안내, "로그아웃 상태"로 전환	PASS	T10_expired.png T10_Alice_expired.png
T11	로그인 후 GET /api/me 로그아웃 후 GET /api/me Alice	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	로그아웃 전 200 / 해당 없음(Authorization 첨부), 로그아웃 후 401 / TOKEN_REQUIRED(Authorization 헤더 없음). 화면 "로그아웃 상태"	PASS	T11_before_logout.png T11_after_logout_headers.png T11_after_logout_response.png
T12	POST /api/posts 본문에 authorId:1, userId:1 을 각각 추가, 전후 GET /api/posts Bob	400 BAD_INPUT, 위조 글 미생성	authorId 400 / BAD_INPUT, userId 400 / BAD_INPUT. 전후 목록 동일	PASS	T12_forged.png
T13	POST /api/posts 에 공백 제목·공백 내용·101 자 제목 각각 전송, 후 목록 조회. 브라우저에서 공백 제목 등록 Alice	400 BAD_INPUT, DB 변경 없음	공백 제목 400 / BAD_INPUT, 공백 내용 400 / BAD_INPUT, 101 자 400 / BAD_INPUT. 목록 변화 없음. 화면 오류 안내, 입력값 유지	PASS	T113_create_invalid.png T13_update_400403.png T13_ui_400_headers.png T13_ui_400_response.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T14	GET /api/posts/99999, GET /api/posts/abc, Alice 토큰으로 PUT·DELETE /api/posts/99999. 브라우저에서 이미 삭제된 글 클릭	없는 글 404, 잘못된 ID 400	조회 404 / NOT_FOUND, abc 400 / BAD_INPUT, 수정 404 / NOT_FOUND, 삭제 404 / NOT_FOUND. 화면 "게시글이 없습니다." 후 목록 자동 갱신	PASS	T14_404400.png T14_ui_404_headers.png T14_ui_404_response.png
T15	POST /api/posts(글 13) → 목록 조회 → 백엔드 종료·재실행 → 목록 조회 Alice	기존 글과 작성자·내용 유지	재시작 전 200 / 해당 없음(2 개), 재시작 후 200 / 해당 없음(같은 2 개, 제목·내용·작성자·createdAt 동일)	PASS	T15_before_restart.png T15_restart.png T15_after_restart
T16	Alice 로그인 후 F5 새로고침	공개 글 유지, 메모리 로그인 상태 해제	GET /api/posts 200 / 해당 없음(Authorization 없음). 글 2 개 유지, 화면 "로그아웃 상태"	PASS	T16_before_refresh.png T16_after_refresh.png
T17	토큰 없이 PUT·DELETE /api/posts/1 (curl) Bob 으로 글 작성 → 수정 → 삭제 (브라우저)	무토큰은 401, Bob 본인 작업은 성공	무토큰 수정 401 / TOKEN_REQUIRED, 삭제 401 / TOKEN_REQUIRED. Bob 작성 201 / 해당 없음(authorId 2), 수정 200 / 해당 없음, 삭제 204 / 해당 없음	PASS	T17_no_token.png T17_bob_create_headers.png T17_bob_create_response.png T17_bob_update_headers.png T17_bob_update_response.png T17_bob_delete_headers.png T17_bob_delete_response.png

HTTP 응답 확인 기록

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	목록·상세 200, 로그인 200, 본인 글 수정 200	T01, T02, T06
새 글 등록	**201 및 Location**	201, Location: /api/posts/2 (Bob 글은 /api/posts/15)	T05, T17
본인 글 삭제	**204 및 빈 본문**	204, Response 탭 'Failed to load response data'(본문 없음)	T06, T17
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	공백 제목·공백 내용·101 자·authorId·userId·ID abc 모두 400 / BAD_INPUT	T12, T13, T14
잘못된 계정	**401 INVALID_CREDENTIALS**	틀린 비밀번호·없는 이메일 모두 401 / INVALID_CREDENTIALS	T03
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	/me·등록·수정·삭제 401 / TOKEN_REQUIRED	T04, T11, T17
변조·클레임 오류	**401 INVALID_TOKEN**	서명 1 글자 변조 토큰으로 4 개 요청 모두 401 / INVALID_TOKEN	T09
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	TTL 3 초, 5 초 후 /me·수정 401 / TOKEN_EXPIRED	T10
다른 작성자의 변경	**403 FORBIDDEN**	Bob → Alice 글 수정·삭제 403 / FORBIDDEN	T07, T08
없는 글	**404 NOT_FOUND**	조회·수정·삭제(99999) 404 / NOT_FOUND	T14

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

WWW-Authenticate: 보호 API 의 401(/me·등록·수정·삭제의 TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED) 응답 헤더에 모두 'WWW-Authenticate: Bearer'가 있었다(Network Response headers 와 curl -i 출력). 자동 검사도 401 마다 이 헤더를 확인하며 통과했다.

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

401 과 403 의 화면 차이: 401 을 받으면 api.js 가 세션을 지워 로그인 칸이 '로그아웃 상태'로 바뀌고 '로그인 시간이 만료되었습니다. 다시 로그인하세요.' 등 code 별 서버 메시지가 표시된다(T10, T11). 403 은 화면에서는 버튼이 숨겨져 발생하지 않도록 했고, 직접 요청으로 받은 403 에서도 세션은 유지된다(코드상 403 은 clearSession 을 호출하지 않음).

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

데이터 불변: 403(T07, T08), 401 INVALID_TOKEN(T09), 401 TOKEN_EXPIRED(T10), 400(T12, T13) 이후 GET 으로 재조회했을 때 글의 title·content·updatedAt 이 요청 전과 같았고 새 글도 생기지 않았다.

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

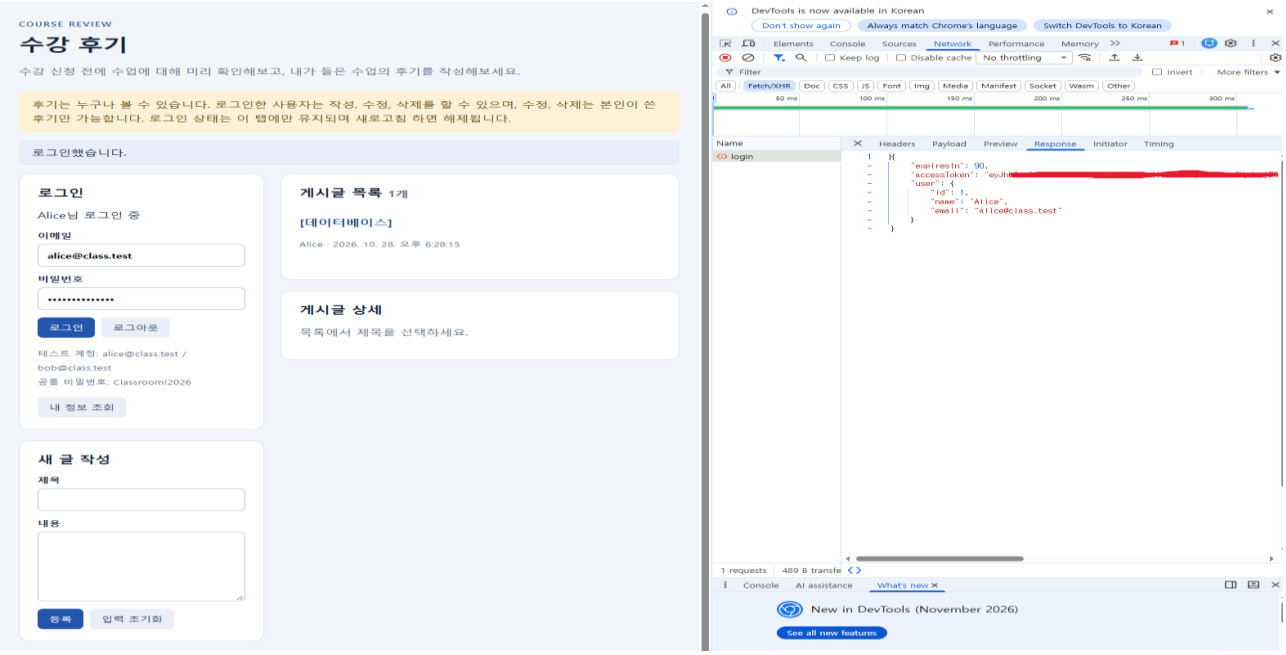
토큰 없이 없는 글 수정: PUT /api/posts/99999 → 401 / TOKEN_REQUIRED. 정상 Alice 토큰으로 같은 요청 → 404 / NOT_FOUND. 인증이 글 존재 확인보다 먼저 수행됨을 확인했다(T14).

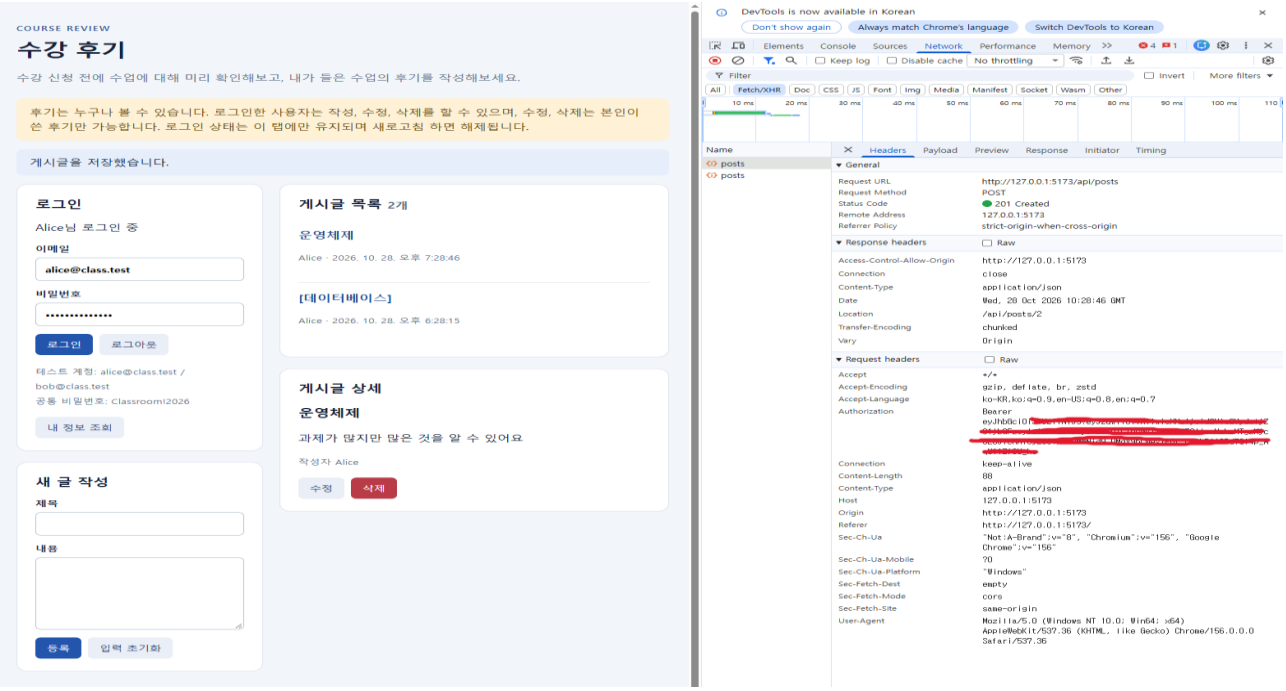
테스트 증거

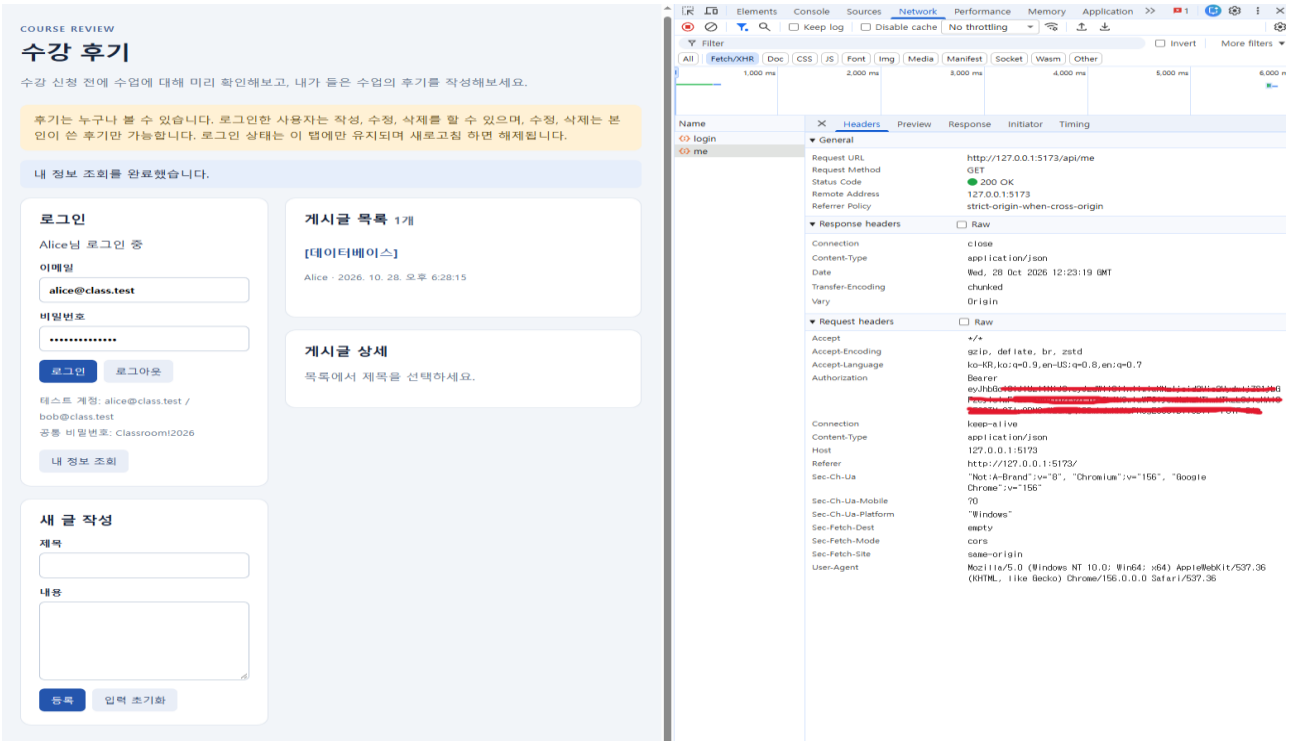
캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

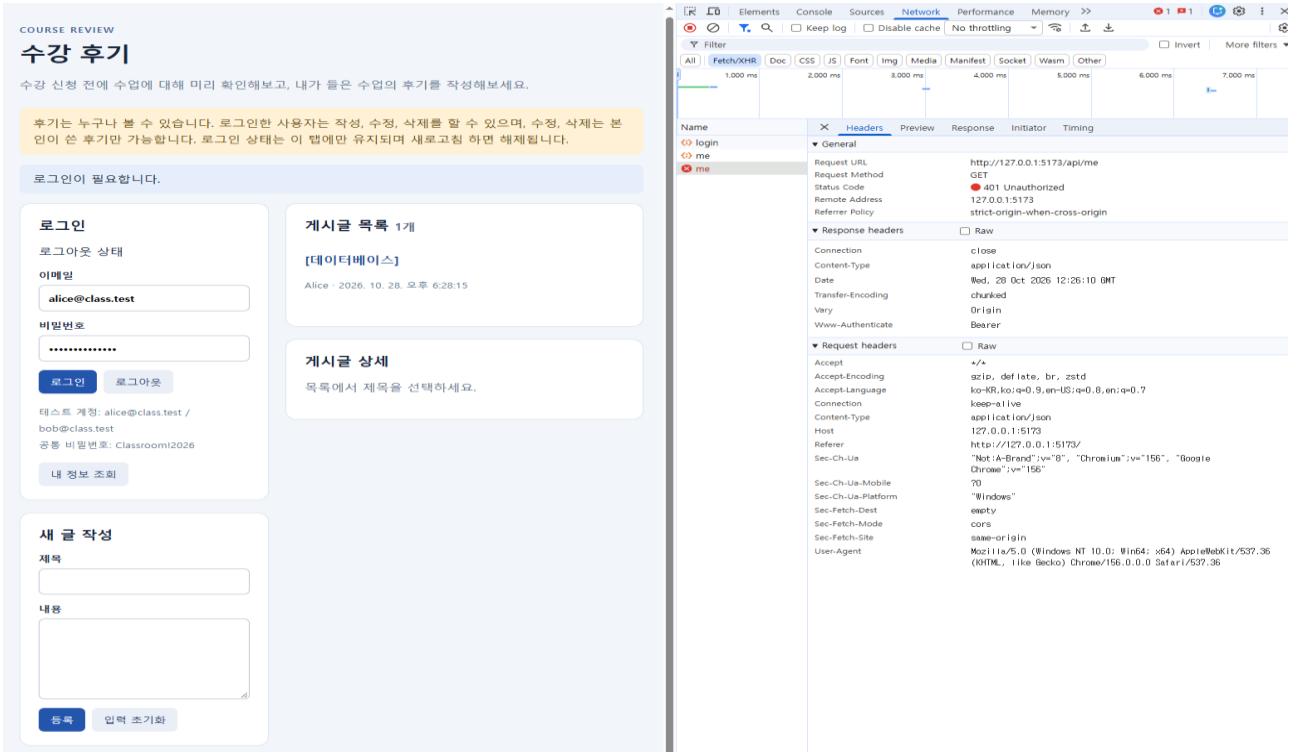
로그인과 토큰 전송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]









다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> function Login($email) { (Invoke-RestMethod -Method Post -Uri "$B/auth/log
in" -ContentType "application/json" -Body ('{"email":"' + $email + '","password":"Classroom!2026"}')).accessToken }
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> curl.exe -i $B/posts/1
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 10:58:50 GMT

{"id":1,"title":"[데이터베이스]","content":"매주 sql실습이 있고 시험 난이도는 적당합니다","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T09:28:15.397480500Z"
,"updatedAt":"2026-10-28T09:28:15.397480500Z"}
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> $Bob = Login "bob@class.test"
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> curl.exe -i -X PUT $B/posts/1 -H "Authorization: Bearer $Bob" -H "Content-
Type: application/json" --data "@update.json"
HTTP/1.1 403
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 10:59:17 GMT

{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정·삭제할 수 있습니다."}
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> curl.exe -i $B/posts/1
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 10:59:24 GMT

{"id":1,"title":"[데이터베이스]","content":"매주 sql실습이 있고 시험 난이도는 적당합니다","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T09:28:15.397480500Z"
,"updatedAt":"2026-10-28T09:28:15.397480500Z"}
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies>
```

```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> $Bob = Login "bob@class.test"

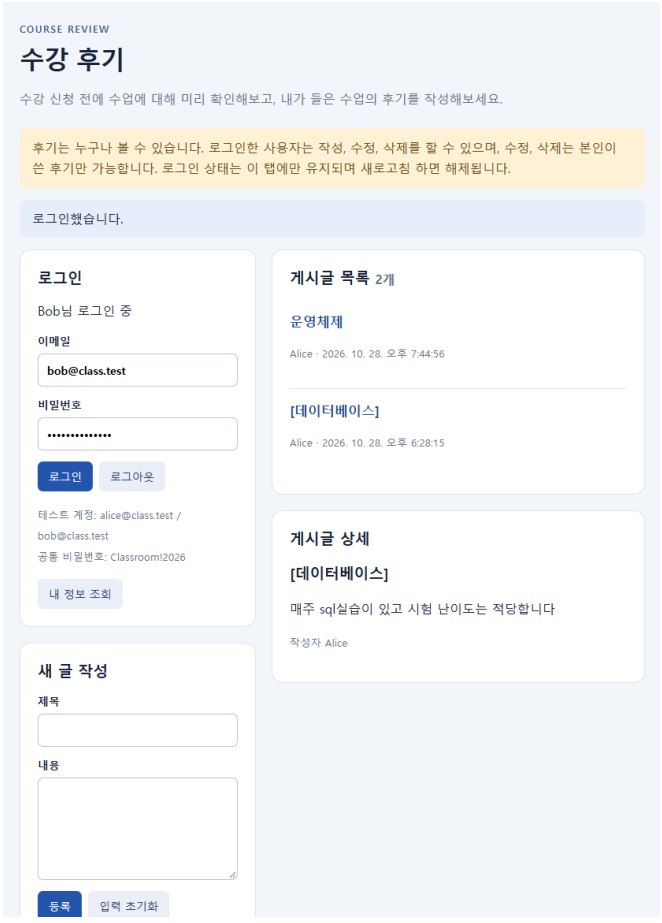
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> curl.exe -i -X DELETE $B/posts/1 -H "Authorization: Bearer $Bob"
HTTP/1.1 403
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 11:06:02 GMT

{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정·삭제할 수 있습니다."}

PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> curl.exe -i $B/posts/1
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 11:06:06 GMT

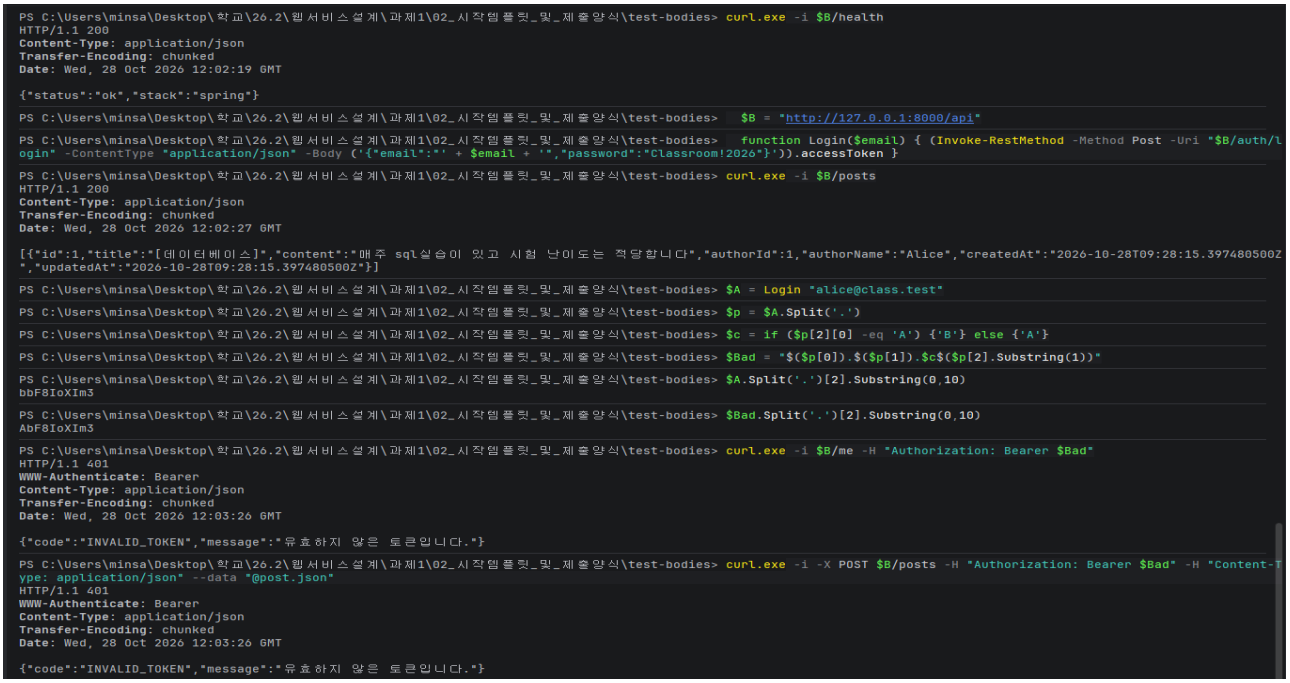
{"id":1,"title":"[데이터베이스]","content":"매주 sql실습이 있고 시험 난이도는 적당합니다","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T09:28:15.397480500Z"
,"updatedAt":"2026-10-28T09:28:15.397480500Z"}

PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies>
```

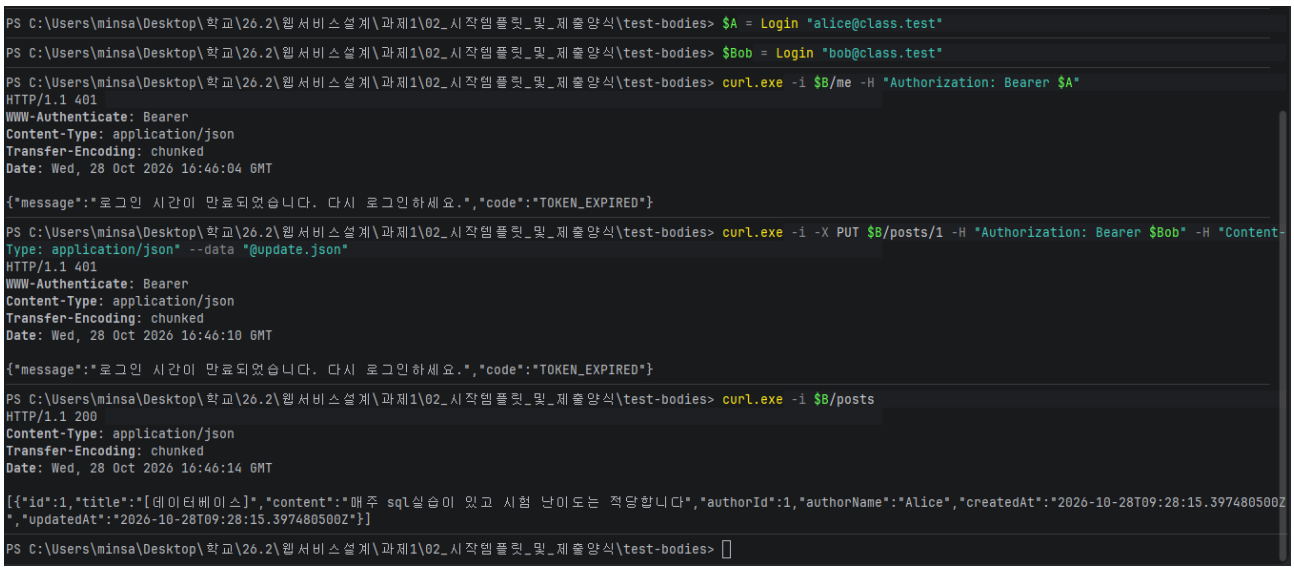


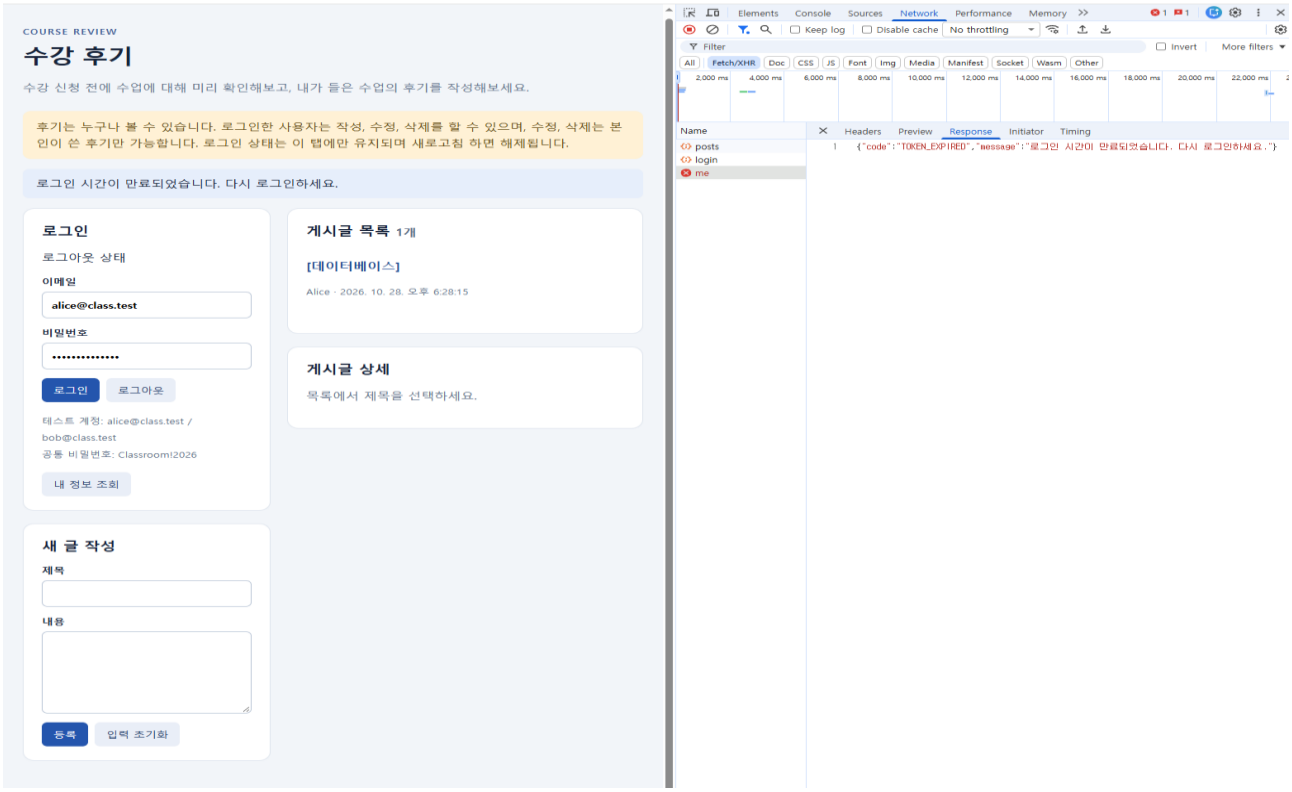
토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]









자동 검사와 로컬 배포 증거

자동 검사: 과제 루트에서 py tools/check_api.py --mode final → 'All 40 checks passed (final)'.
TOKEN_TTL=3 으로 재시작 후 --expiry 추가 → 만료 토큰 GET·POST·PUT·DELETE 401 TOKEN_EXPIRED 포함 'All 48 checks passed (final)'. 이 도구는 API 응답만 확인하므로 화면·재시작 확인은 T10·T11·T14·T15·T16 에서 수동으로 수행했다.

```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\JWT-Assignment> py tools/check_api.py --mode final
PASS health
PASS public list
PASS bad ID
PASS missing ID
PASS wrong password
PASS unknown email
PASS no token me
PASS no token create
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/20
PASS tampered DELETE /posts/20
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
```

```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\JWT-Assignment> py tools/check_api.py --mode final
PASS health
PASS public list
PASS bad ID
PASS missing ID
PASS wrong password
PASS unknown email
PASS no token me
PASS no token create
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/20
PASS tampered DELETE /posts/20
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
```

```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\JWT-Assignment\frontend-vue> npm run build

> jwt-board-vue@1.0.0 build
> vite build

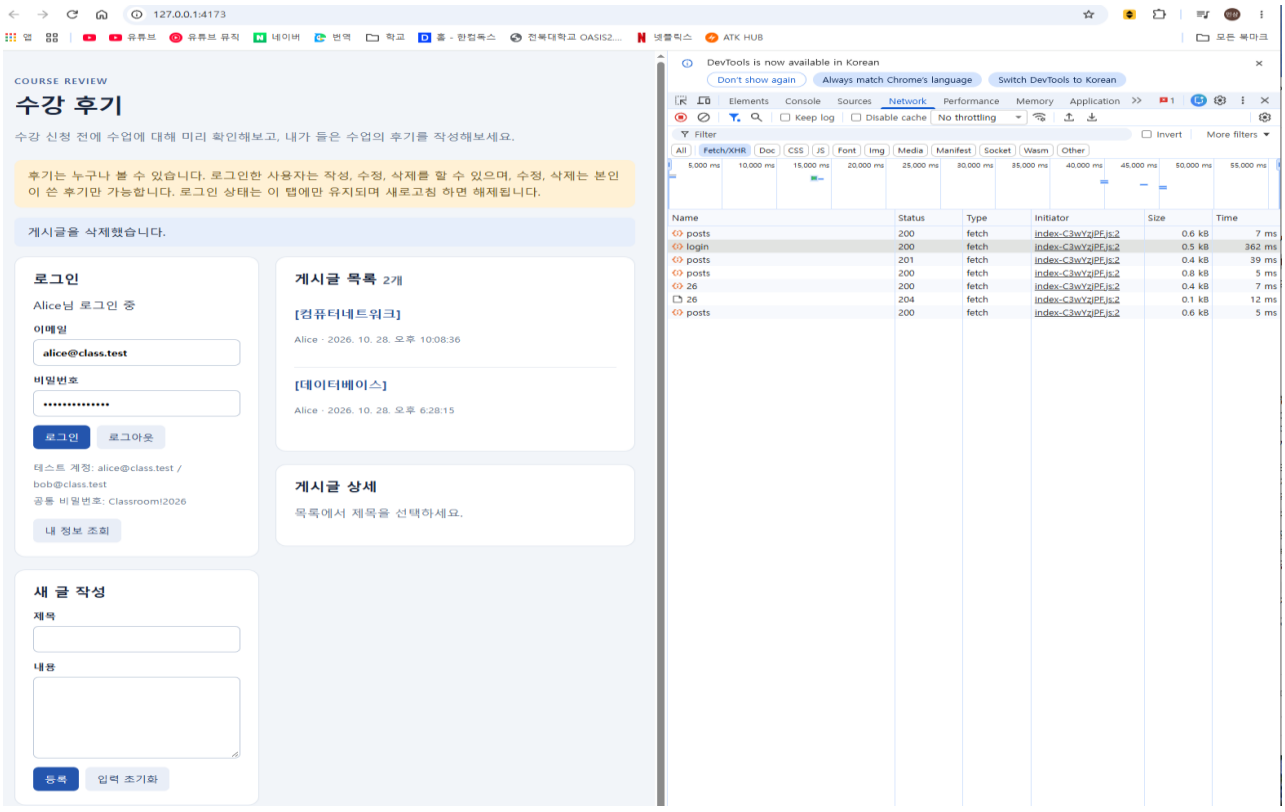
vite v8.3.4 building client environment for production...
✓ 12 modules transformed.
computing gzip size...
dist/index.html                  0.42 kB │ gzip:  0.31 kB
dist/assets/index-DaISom2Y.css   1.65 kB │ gzip:  0.79 kB
dist/assets/index-C3wYzjPF.js   69.47 kB │ gzip: 27.55 kB

✓ built in 116ms
```

```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\JWT-Assignment\frontend-vue> npm run preview

> jwt-board-vue@1.0.0 preview
> vite preview --host 127.0.0.1

  ➜  Local:   http://127.0.0.1:4173/
  ➜  press h + enter to show help
```



```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> $A = Login "alice@class.test"; curl.exe -i -X POST $B/posts -H "Authorizat
ion: Bearer $A" -H "Content-Type: application/json" --data "@post.json"
HTTP/1.1 201
Location: /api/posts/13
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 13:08:36 GMT

{"id":13,"title":"[컴퓨터네트워크]","content":"패킷 분석 실습이 재미있고 과제는 매주 있습니다.","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T13:08:36.96389
3700Z","updatedAt":"2026-10-28T13:08:36.963893700Z"}

PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> curl.exe -i $B/posts
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 13:08:39 GMT

[{"id":13,"title":"[컴퓨터네트워크]","content":"패킷 분석 실습이 재미있고 과제는 매주 있습니다.","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T13:08:36.9638
93700Z","updatedAt":"2026-10-28T13:08:36.963893700Z"},{"id":1,"title":"[데이터베이스]","content":"매주 sql실습이 있고 시험 난이도는 적당합니다","authorId":1,"authorName
":"Alice","createdAt":"2026-10-28T09:28:15.397480500Z","updatedAt":"2026-10-28T09:28:15.397480500Z"}]

PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies>
```

```
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  46:35 min
[INFO] Finished at: 2026-10-28T22:09:13+09:00
[INFO] ------------------------------------------------------------------------
일괄 작업을 끝내시겠습니까 (Y/N)? y
```

```
PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies> curl.exe -i $B/posts
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 28 Oct 2026 13:10:10 GMT

[{"id":13,"title":"[컴퓨터네트워크]","content":"패킷 분석 실습이 재미있고 과제는 매주 있습니다.","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T13:08:36.9638
93700Z","updatedAt":"2026-10-28T13:08:36.963893700Z"},{"id":1,"title":"[데이터베이스]","content":"매주 sql실습이 있고 시험 난이도는 적당합니다","authorId":1,"authorName
":"Alice","createdAt":"2026-10-28T09:28:15.397480500Z","updatedAt":"2026-10-28T09:28:15.397480500Z"}]

PS C:\Users\minsa\Desktop\학교\26.2\웹서비스설계\과제1\02_시작템플릿_및_제출양식\test-bodies>
```

AI 활용과 참고 자료

AI 사용 여부: [사용 / 미사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude	코드생성	AuthService.java	수정 후 반영
Claude	코드생성	api.js	수정 후 반영
Claude	테스트	4 장 테스트	테스트에 사용

대표 사례

사례 1

도구 / 날짜: Claude / 2026-10-27

요청한 내용: 브라우저에서 보낼 수 없는 요청을 curl 로 시험하는 방법을 요청

AI 가 제안한 내용: curl.exe 명령과 요청 본문 JSON 파일(test-bodies),

실제 반영 위치: 보고서 4 장 테스트

내가 확인하고 수정한 내용: 요청 본문 제목을 앞서 만든 형식과 동일하게 맞춤

검증 방법과 결과: 각 파일로 보낸 요청의 상태 코드와 code 를 curl.exe -i 출력에서 직접 확인했다. update.json 으로 Bob 이 Alice 글을 수정하면 403 FORBIDDEN 이었고, 재조회한 글에 "(수정 시도)"가 없어 원문 유지를 확인했다(T07). forged 파일 두 개는 각각 400 BAD_INPUT(T12), blank·too_long 파일은 400 BAD_INPUT(T13)이었으며 요청 후 목록이 바뀌지 않았다.

사례 2

도구 / 날짜: Claude / 2026-10-28

요청한 내용: py tools/check_api.py –mode final 입력 시 오류가 발생 하여 질문

AI 가 제안한 내용: Python 이 설치되지 않은 상태라 설명하고 winget 설치 방법을 알려줌

실제 반영 위치: 보고서 4 장 자동검사

내가 확인하고 수정한 내용: winget 으로 Python 을 설치

검증 방법과 결과: 새 터미널에서 py tools/check_api.py –mode final 을 실행하여 정상 작동하는 것을 확인

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 6 주차 JWT 로그인과 인증, 7 주차 JWT 접근제어와 Swagger 강의자료	AuthService.issueToken, currentUserId	JJWT 발급·검증 흐름(verifyWith·requireIssuer·requireAudience)과 assertOwner 개념 참고

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId 를 믿으면 Bob 이 authorId:1 을 넣어 Alice 이름으로 글을 쓰거나, 작성자를 바꿔 남의 글을 자기 글로 만들 수 있다. 내 서버는 작성자를 토큰 검증 결과(currentUserId)로만 정하고, 제공 코드 fields()가 title·content 외의 필드를 400 BAD_INPUT 으로 거부한다. T12 에서 authorId·userId 를 넣은 요청은 400 이었고 글이 생성되지 않았다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼 숨김은 브라우저 화면에서만 동작하므로, curl 이나 개발자 도구로 PUT·DELETE 를 직접 보내면 우회된다. 실제 권한 검사는 서버의 AuthService.assertOwner 가 하며, BoardController.update·delete 가 DB 를 바꾸기 전에 호출한다. T07·T08 에서 버튼이 없는 Bob 이 curl 로 보낸 요청은 403 이었다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰 문자열을 Base64 로 풀어 내용을 읽기만 하므로, 누군가 payload 의 sub 를 바꾸거나 서명을 조작해도 알아채지 못한다. verify 는 서버 키로 서명을 다시 계산해 비교하고 만료·발급자·대상까지 확인한다. 내 코드는 parseSignedClaims 로 서명(HS256, 서버 키), exp 만료, iss, aud 를 검증하고, 헤더 알고리즘이 HS256 인지와 exp 존재, sub 형식과 실제 계정 존재를 추가로 확인한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 이 탭의 메모리에서 토큰과 사용자 정보를 지워, 이후 요청에 헤더가 붙지 않게 한다. 만료는 서버가 exp 가 지난 토큰을 401 TOKEN_EXPIRED 로 거부하는 것이고, 프론트는 이 401 을 받아 로그인 상태를 해제한다. 새로고침은 페이지 JS 가 다시 시작되며 메모리 변수가 초기화되어 로그인이 풀린다(글은 DB 에 있어 유지). 이미 복사된 토큰은 로그아웃과 무관하게 exp 까지 서버에서 유효하다. 서버에 토큰 차단 목록이 없기 때문이다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

서버를 재시작한 뒤 화면 목록이 0 개로 보여 데이터가 지워진 줄 알았던 문제
원인 확인: curl 로 GET /api/posts 를 직접 조회하니 글이 남아 있었음 → 서버가 꺼진 동안 화면이 목록 요청에 실패했고, 로그인은 목록을 다시 불러오지 않아 빈 화면이 유지된 것
조치: 새로고침
재검증: T15 에서 재시작 전후 목록 비교

현재 남은 문제와 다음에 개선할 점: 없음